FitPick – Digital Wardrobe & Outfit Planner

Software Design Document

Digital Assignment 2 / Review 2 · Software Engineering

Author: Nithilan A (nithi0307) · Repository: github.com/nithi0307/fitpick · Design files: docs/design/

1. Project Overview

FitPick is a web application that lets users organise the clothes they own, search and filter their wardrobe, combine items into outfits, save favourites and manage a simple profile. It is built as a **React + Vite** single-page frontend, a **FastAPI** REST backend and a **PostgreSQL 16** database, all started together with **Docker Compose**. The requirements from Assignment 1 – 25 user stories prioritised with MoSCoW and six Figma screens – are the input to this design.

This document explains how the design applies **abstraction, modularity, cohesion and low coupling**, presents the high-level architecture and the six UI screens, lists the key design decisions with their reasoning, and shows short code snippets from the repository that demonstrate each principle. The goal of this version is a clean, maintainable structure rather than a large feature set: no AI recommendations, authentication tokens or micro-services are claimed.

2. Design Principles Applied

2.1 Abstraction

Each layer of FitPick exposes a small, stable interface and hides how it works. On the frontend, pages never call `fetch` or know the backend URL; they call named functions such as `getClothingItems()` from `services/api.js` and receive plain JavaScript objects. On the backend, the routers never touch storage directly; they talk to a `ClothingRepository` that offers `list / get / add / delete`. The repository currently stores items in memory, and the PostgreSQL schema is defined in `database/schema.sql`; because the routers depend only on the repository's methods, connecting the database later changes one class and no endpoint.

frontend/src/services/api.js – pages depend on these functions, not on HTTP details

```
const BASE_URL = import.meta.env.VITE_API_URL ?? "http://localhost:8000";

async function request(path, options = {}) {
  const response = await fetch(`${BASE_URL}${path}`, {
    headers: { "Content-Type": "application/json" },
    ...options,
  });

  if (!response.ok) {
    throw new Error(`Request failed: ${response.status}`);
  }

  return response.json();
}

export function getClothingItems() {
  return request("/clothing");
}
```

2.2 Modularity

The code base is divided into small units that can be understood and changed independently. The frontend has one file per screen in `pages/` (six files, matching the six Figma screens), one file per reusable widget in `components/` and a single API module in `services/`. The backend mirrors this with one router per feature area

(`routers/clothing.py`, `routers/outfits.py`), one schema file and one repository per entity. Adding a new feature such as "Outfit History" means adding a page, a route entry, a router and a repository – existing modules are not edited.

backend/main.py – each feature is a self-contained router plugged in with one line

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from routers import clothing, outfits
app = FastAPI(title="FitPick API", version="0.1.0")
app.include_router(clothing.router)
app.include_router(outfits.router)
```

2.3 Cohesion

Every module has one clear job. `Sidebar.jsx` only renders navigation links; `Navbar.jsx` only shows the page title and user; `Button.jsx` only renders a styled button and knows nothing about what happens on click. On the backend, `schemas.py` contains only data shapes, `routers/clothing.py` contains only HTTP handling for clothing (status codes, 404 errors), and `repositories/clothing_repository.py` contains only storage logic. The database schema is equally focused: `users`, `clothing_items`, `outfits` and the `outfit_items` join table each store one concept. High cohesion makes each file short (most are under 60 lines) and easy to test in isolation.

2.4 Low Coupling

Modules communicate only through explicit interfaces, so a change in one rarely forces a change in another. The frontend and backend are coupled only by the JSON shapes defined in `schemas.py` – the backend can change its storage, and the frontend can change its styling, without touching the other side. Inside the frontend, pages are rendered through React Router's `<Outlet />` so `Layout` does not import any page and pages do not import the sidebar. Components receive everything they need as props (`variant`, `title`, `onClick`). Inside the backend, routers import a repository instance but never the data structure inside it. Docker Compose adds infrastructure decoupling: each tier is a separate container that talks to the others over the network by service name.

frontend/src/components/Layout.jsx – the shell renders pages through <Outlet /> and never imports them

```
export default function Layout() {
  return (
    <div className="layout">
      <Sidebar />
      <div className="main">
        <Navbar />
        <main className="content">
          <Outlet />
        </main>
      </div>
    </div>
  );
}
```

3. High-Level Architecture

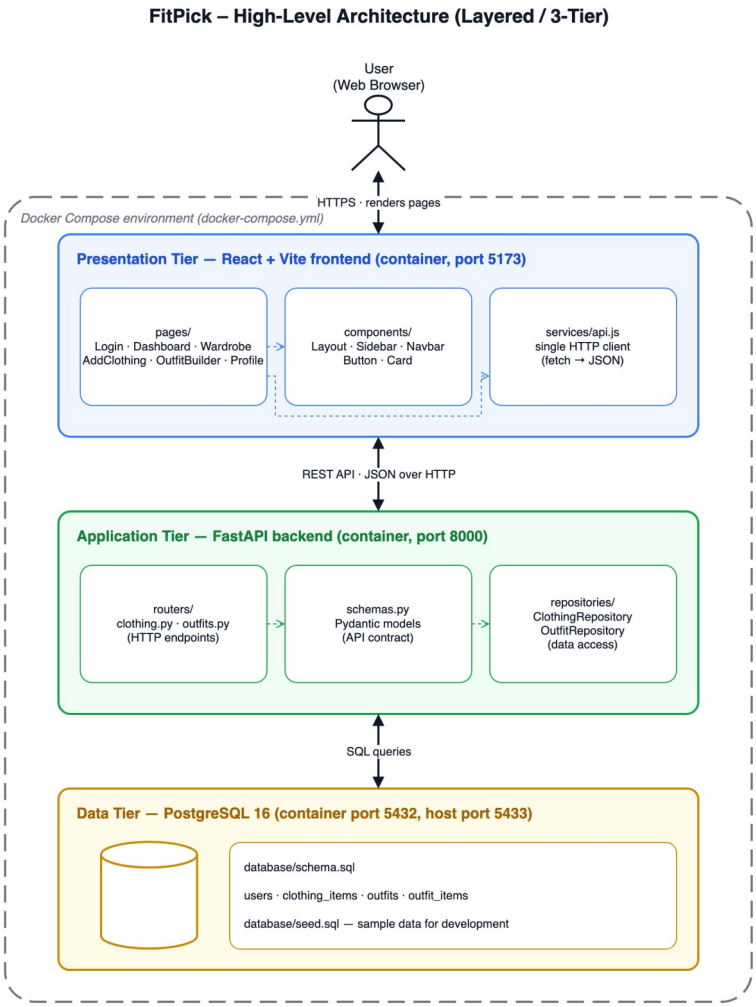


Figure 1 – FitPick high-level architecture (source: docs/design/architecture/fitpick-architecture.drawio)

3.1 Architecture style: layered (three-tier)

FitPick follows a **layered / three-tier architecture**. The **presentation tier** is the React single-page application that renders the six screens in the browser. The **application tier** is the FastAPI backend, which exposes a REST API (JSON over HTTP) and contains all business rules. The **data tier** is PostgreSQL, holding users, clothing items and outfits. Calls only go downwards: the browser talks to the API, the API talks to the database, and no tier skips a level. All three run as separate containers defined in `docker-compose.yml` (frontend 5173, backend 8000, database 5432 inside the container mapped to 5433 on the host).

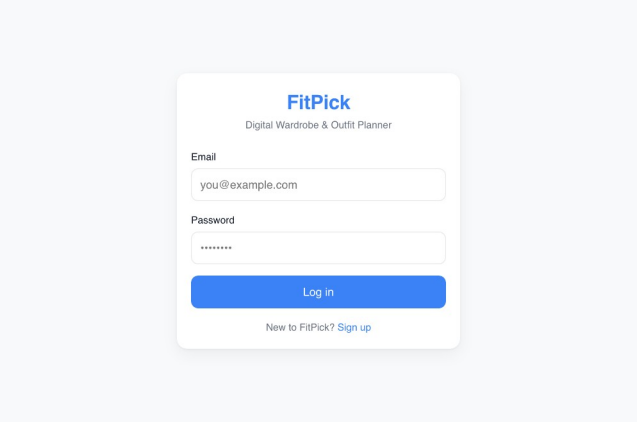
3.2 Why this style

- **Fits the problem.** FitPick is a classic CRUD application (create, read, update, delete wardrobe data). A layered design is the standard, proven fit for that shape and does not need the operational overhead of micro-services.
- **Separation of concerns.** UI, business logic and storage each live in one tier, which is exactly what the four design principles in Section 2 require.
- **Independent evolution.** The React app can be restyled, the API can gain endpoints and the database can be tuned separately. The repository layer inside the backend even allows the storage mechanism to change without changing the API.
- **Simple to run and explain.** One developer can start the whole system with `docker compose up`, and a newcomer can understand the flow from the single diagram above.

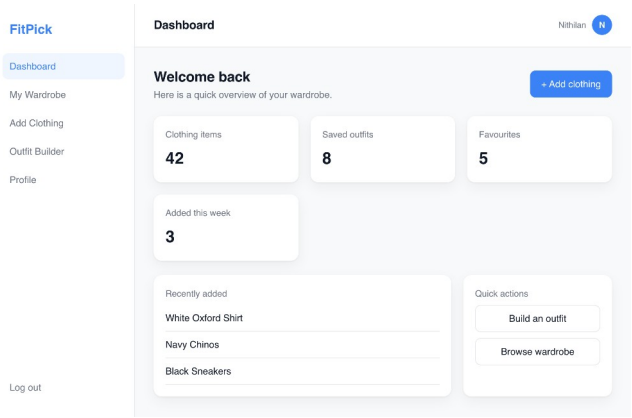
- **Room to grow.** Authentication, image upload or outfit suggestions can be added as new routers and pages without restructuring the system.

4. User Interface Design

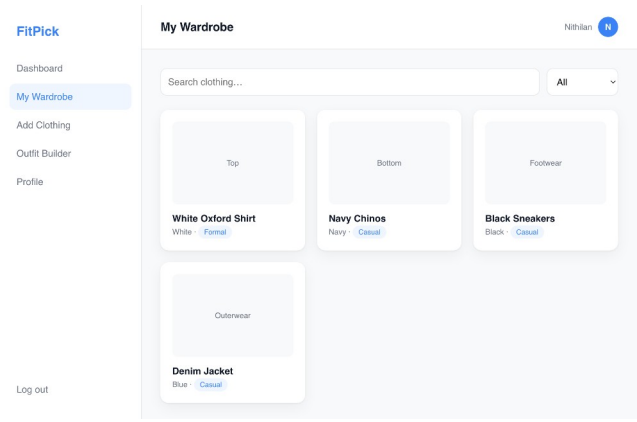
The six screens were designed in Figma and then implemented as six React pages with the same names; the screenshots below are the implemented screens running in the browser, which are the improved versions of the Figma designs. They share one design system: royal blue #3B82F6 for primary actions and the active navigation item, white cards with rounded corners and soft shadows on a light grey #F8F9FA background, and one clean sans-serif typeface.



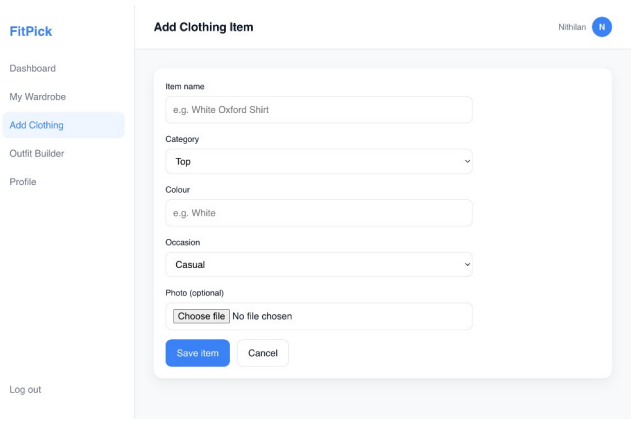
1. Login / Sign Up



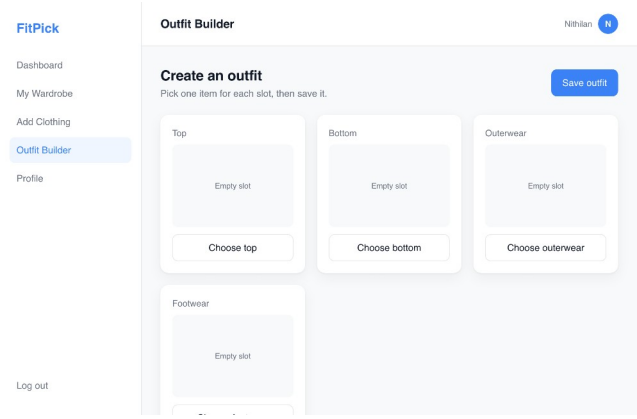
2. Dashboard



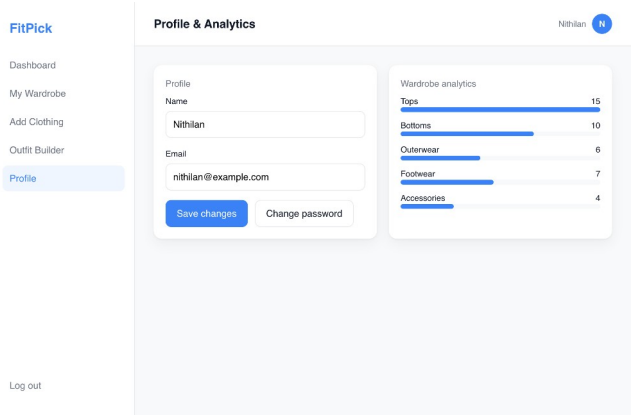
3. My Wardrobe



4. Add Clothing Item



5. Outfit Builder



6. Profile & Analytics

Figure 2 – The six FitPick screens (designed in Figma, implemented in React). Files: docs/design/wireframes/

4.1 How the UI was made user-friendly

- **Consistent navigation.** Every screen after login has the same left sidebar with the five main sections and a log-out link, and the same top bar showing where the user is. Nothing moves between screens, so users never have to re-learn the layout.
- **One primary action per screen.** Each screen has a single blue button for its main task – "Log in", "+ Add clothing", "Save item", "Save outfit", "Save changes" – and grey secondary buttons for everything else, so the next step is always obvious.
- **Cards for wardrobe content.** Clothing items, outfit slots and statistics are shown as uniform cards with a preview area, a name, and small tags for colour and occasion. The same card is used on the Dashboard, Wardrobe and Outfit Builder, so users recognise items instantly.
- **Search and filter at the top.** On My Wardrobe the search box and category filter sit above the grid and react as the user types, which supports the "find an item in seconds" success metric.
- **Short, labelled forms.** Add Clothing and Profile use clearly labelled fields with placeholder examples ("e.g. White Oxford Shirt"), drop-downs for fixed choices such as category and occasion, and an optional photo – the minimum data needed, nothing more.
- **Clear visual hierarchy.** Page titles are large and dark, helper text is smaller and grey, and numbers on statistic cards are bold, so the eye lands on the important information first.
- **Minimal clutter and readable text.** Generous spacing, one accent colour and no decorative elements keep every screen calm; the login screen contains only the brand, two fields and one button.
- **Feedback states.** The Wardrobe shows "Loading..." while fetching and a plain-language message if the backend cannot be reached, so the user is never left with a blank screen.

5. Design Decisions & Why

Decision	Why
Layered 3-tier architecture (React → FastAPI → PostgreSQL)	Each tier can be developed, tested and replaced on its own. The UI never touches SQL, and the database never knows about React. This is the simplest structure that still enforces separation of concerns for a one-developer project.
React with one component per screen and a small set of shared components	The six Figma screens map one-to-one onto six page files, so the design and the code stay in sync. Layout, Sidebar, Navbar, Button and Card are written once and reused, which removes duplication and keeps every screen visually identical.
FastAPI split into routers, schemas and repositories	Routers handle only HTTP, Pydantic schemas define the contract, and repositories hide storage. Swapping the in-memory repository for a PostgreSQL one changes a single class and no endpoint code.
PostgreSQL with a normalised schema	Users, clothing items and outfits are naturally relational (an outfit is a set of items owned by a user). Foreign keys and a join table keep the data consistent, and a relational DB is the default for structured CRUD applications.
Docker Compose for the whole environment	One command starts frontend, backend and database with the same versions on any machine, which removes "works on my laptop" problems and keeps the port mapping (5433 → 5432) documented in one file.

6. Code Structure & Snippets

The repository layout mirrors the architecture: each tier is a top-level folder and each design principle can be seen in the file structure itself.

Repository layout (see README.md for the full tree)

```
fitpick/
├── frontend/src/
│   ├── App.jsx                  # route table
│   ├── pages/                   # six screens: Login ... Profile
│   ├── components/              # Layout, Sidebar, Navbar, Button, Card
│   ├── services/api.js          # only module that talks to the backend
│   └── styles/global.css        # design tokens (colours, radius, shadow)
├── backend/
│   ├── main.py                  # app factory: CORS + include_router
│   ├── routers/                 # clothing.py, outfits.py  (HTTP only)
│   ├── schemas.py               # Pydantic models = API contract
│   └── repositories/            # ClothingRepository, OutfitRepository (storage only)
├── database/schema.sql, seed.sql
├── docs/design/                 # .drawio, PNG, screen images, this PDF
└── docker-compose.yml
```

frontend/src/App.jsx – routing: Login stands alone, every other page is nested under Layout

```
export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Login />} />

        <Route element={<Layout />}>
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/wardrobe" element={<Wardrobe />} />
          <Route path="/add-clothing" element={<AddClothing />} />
          <Route path="/outfit-builder" element={<OutfitBuilder />} />
          <Route path="/profile" element={<Profile />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

frontend/src/components/Button.jsx – one reusable button; the variant prop selects the style

```
export default function Button({
  children,
  variant = "primary",
  block = false,
  type = "button",
  onClick,
}) {
  const classes = ["btn", `btn--${variant}`, block ? "btn--block" : ""]
    .filter(Boolean)
    .join(" ");

  return (
    <button type={type} className={classes} onClick={onClick}>
      {children}
    </button>
  );
}
```

backend/routers/clothing.py – the router translates HTTP to repository calls and back; no storage logic

```
from fastapi import APIRouter, HTTPException

from repositories.clothing_repository import ClothingRepository
from schemas import ClothingItem, ClothingItemCreate

router = APIRouter(prefix="/clothing", tags=["clothing"])
repository = ClothingRepository()

@router.get("", response_model=list[ClothingItem])
def list_clothing():
    return repository.list()

@router.post("", response_model=ClothingItem, status_code=201)
def create_clothing(data: ClothingItemCreate):
    return repository.add(data)
```

backend/repositories/clothing_repository.py – the storage interface the routers depend on

```
class ClothingRepository:
    ...
    def list(self) -> list[ClothingItem]:
        return list(self._items.values())

    def add(self, data: ClothingItemCreate) -> ClothingItem:
        item = ClothingItem(id=self._next_id, **data.model_dump())
        self._items[item.id] = item
        self._next_id += 1
        return item
```

database/schema.sql – relational model: an outfit is a set of clothing items owned by a user

```
CREATE TABLE IF NOT EXISTS clothing_items (
    id          SERIAL PRIMARY KEY,
    user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name        VARCHAR(100) NOT NULL,
    category    VARCHAR(50) NOT NULL,   -- Top, Bottom, Outerwear, Footwear, Accessory
    colour      VARCHAR(50) NOT NULL,
    occasion    VARCHAR(50) NOT NULL,   -- Casual, Formal, Sport, Party
    image_url   TEXT,
    is_favourite BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS outfit_items (
    outfit_id        INTEGER NOT NULL REFERENCES outfits(id) ON DELETE CASCADE,
    clothing_item_id INTEGER NOT NULL REFERENCES clothing_items(id) ON DELETE CASCADE,
    PRIMARY KEY (outfit_id, clothing_item_id)
);
```

7. Maintainability & Future Changes

The structure above was chosen so that the most likely future changes are cheap:

- **Connecting PostgreSQL.** Implement the same four methods of `ClothingRepository` with SQL queries against `schema.sql`. Routers, schemas and the frontend do not change.
- **Adding a screen** (e.g. Outfit History): create `pages/OutfitHistory.jsx`, add one <Route> in `App.jsx` and one entry in the Sidebar link list; the layout, navbar and styling are inherited automatically.
- **Adding an API feature** (e.g. Change Password): create `routers/users.py` and register it with one `include_router` line.
- **Restyling.** Colours, radius and shadow are CSS variables in `global.css`; changing the accent colour is a one-line edit that updates every button, link and tag.
- **Testing.** Because routers depend on a repository object, endpoint tests can use the in-memory repository without a database; React components can be tested in isolation because they receive data through props.

8. Conclusion

FitPick's design turns the Assignment 1 requirements into a small, clearly layered system. Abstraction is provided by the API service module and the repository classes, modularity by one-file-per-page and one-router-per-feature, cohesion by giving every file a single responsibility, and low coupling by connecting modules only through props, JSON schemas and the network. The Draw.io diagram, the six Figma screens and all editable sources are committed under `docs/design/` in the GitHub repository, and the README's "Software Design" section links to them.